

Structural Design Patterns in C#

The Structural Design Patterns in C# focus on how classes and objects can be composed to form larger structures. These patterns are used to manage the relationships between entities efficiently. In this article, I will give a brief introduction to Structural Design Patterns in C#. Please read our previous section articles, where we discussed the [Creational Design Patterns in C#](#) with examples. As part of this article, we are going to discuss the following three things.

1. [What is the Structural Design Pattern?](#)
2. [When to use the Structural Design Pattern?](#)
3. [Examples of Structural Design Patterns](#)
4. [Benefits of Structural Design Patterns in C#:](#)

What is the Structural Design Pattern in C#?

The Structural Design Patterns simplify the design by identifying a simple way to manage the relationships between entities. They allow developers to obtain new functionalities by composing objects and classes. They focus on how classes inherit from each other and how they are composed from other classes.

When to use Structural Design Patterns in C#?

In real-time applications, sometimes we need to change the structure of a class or the relationship among the classes, but we don't want this change to be affected by the project. For example, if we have two classes, let's say User and Product. And the Product class is used inside the User class making one-to-many relationships between the User and Product. Tomorrow, the structure or the relationships between these two classes will change. The customer now wants to keep away the Product class from the User class, as they want to use the User and Product class independently. This is actually a structural change and we don't want this structural change to affect our project. This is where the Structural Design Pattern helps us.

Examples of Structural Design Patterns in C#:

The following is the list of Structural Design Patterns.

- **Adapter Pattern:** The [Adapter Design Pattern](#) allows objects with incompatible interfaces to collaborate. It's commonly used when existing classes need to be used and modifying their interfaces isn't an option, such as when dealing with third-party or legacy systems.
- **Bridge Pattern:** The [Bridge Design Pattern](#) separates an abstraction from its implementation so that the two can vary independently. This is useful when both the interface and underlying implementations can have multiple variants and should be interchangeable.
- **Composite Pattern:** The [Composite Design Pattern](#) allows us to compose objects into tree structures to represent part-whole hierarchies. It lets clients treat individual objects and compositions uniformly, ideal for graphical applications or file systems where objects might contain other objects.
- **Decorator Pattern:** The [Decorator Design Pattern](#) adds new functionalities to objects dynamically by placing these objects inside special wrapper objects that contain the behaviors. This is useful in C# when we want to add features to objects without subclassing.
- **Facade Pattern:** The [Facade Design Pattern](#) provides a simplified interface to a complex subsystem. It is used to provide a cleaner API over a set of interfaces in a subsystem.
- **Flyweight Pattern:** The [Flyweight Design Pattern](#) minimizes memory use by sharing as much data as possible with similar objects. It is useful when a program requires a large number of objects with similar states.
- **Proxy Pattern:** The [Proxy Design Pattern](#) provides a surrogate or placeholder for another object to control access to it. This is useful for managing operations on expensive resources or operations that require security.

Benefits of Structural Design Patterns

The following are the Benefits of Structural Design Patterns in C#:

- **Scalability:** Facilitates more scalable and dynamic system architectures.
- **Efficiency:** This reduces the operational overhead in applications by ensuring that only necessary objects are in memory and being managed or used.
- **Flexibility:** Increases the system's flexibility by decoupling its components, which can develop independently but still operate together.

In the next article, I will discuss the [Adapter Design Pattern in C#](#) with Examples. In this article, I try to give a brief introduction to Structural Design Patterns. I hope you understand the need and use of Structural Design Patterns in C#.